Meeting ?? SP23

CoLT

@nebu

Outline

Virtual Memory
- What is Virtual Memory?
- How is it Implemented?
- Contiguity for TLB Reach
- Understanding Sources of Contiguity

Exploiting Intermediate Contiguity: CoLT (Coalesced Large-Reach TLBs)
- Design and Implementation
- Benchmarks

Section 1

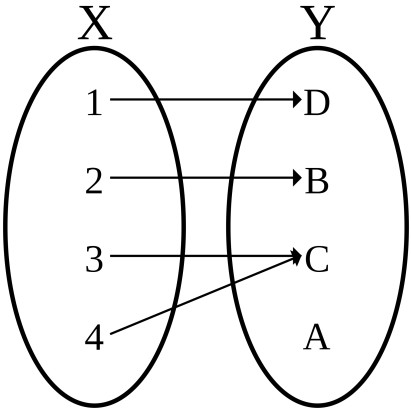
Virtual Memory

Subsection 1

What is Virtual Memory?

Is a Mapping From "Virtual" Memory Addresses to Physical Addresses

- Neither injective nor surjective.



Why?

- Provides each process with the illusion of a large and private address space.
- etc.

Subsection 2

How is it Implemented?

Pages and Frames

- The OS partitions the virtual address space into “pages” (chunks of memory of some size).
- Maps the “pages” to physical “frames”.

Translation

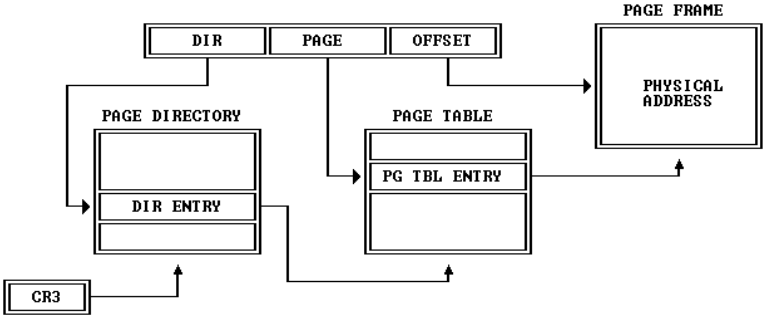
- The mapping is stored in a “page table”.
- Page table consists of PTEs, which contain:
  - Actual mapping
  - Bookkeeping bits
- The actual translation is performed by the CPU at runtime by referring to the page table.

Page Tables in the Real World

- Are (naturally) more complex.
- x86 implements a 4-level radix tree.
- To do address translation, it requires hardware to walk the page table tree and find the physical frame.

Diagram

Figure 5-9. Page Translation



Page Table Size

- x86-64's base page size is 4KB.
- Huge pages of 2MB and 1GB also supported.
- OS support: Linux Transparent Huge Pages (THP).

Translation Lookaside Buffer

- So, a single memory access with a virtual address requires 4 more memory accesses to walk the page table tree? Bad.
- Cache the translations!

TLB

- TLB hit reduces translation step to a few cycles instead of multiple memory accesses.
- TLB miss still requires ~hundreds of cycles.

TLB Interaction with Varied Page Sizes

- LSBs of virtual address used as offset into page.
- Bits above page offset used as index into TLB.

TLB Interaction with Varied Page Sizes

- For different page sizes, different number of offset bits.
- But page size is unknown at lookup time.
- Which bits do you use to index into TLB?
- Even “simple” paging schemes create hard problems!

MMU Caching

- Effectively caches recently used PTEs from tree hierarchy to prevent having to access the tree from the root.
- Reduces the number of memory accesses required on a miss.

OS Side

- Buddy allocator: Tracks free physical memory, merges and splits blocks to minimize fragmentation.
- Demand paging/lazy allocation: OS only sets up PTE at first memory access to that location (handles a page fault).
- Memory compaction: Reduces external fragmentation by "compacting" scattered physical memory to larger, contiguous blocks of physical memory.

Subsection 3

Contiguity for TLB Reach

Bigger Better Faster More!

- We're always trying to make TLBs more effective to reduce the overhead of virtual memory, since it can degrade application performance by…a lot.[1]
- Lots of research in TLB prefetching, faster page walks.
- Orthogonal to this is the idea of trying to use a single TLB entry for more translations.

[1] C. McCurdy, A. Cox, and J. Vetter, "Investigating the TLB Behav- ior of High-End Scientific Appplications on Commodity Multipro- cessors," ISPASS, 2008, etc.

Huge Pages/Superpages

- Large page tables are nice, because they increase “TLB reach”.
- They’re usually a power-of-two times the base page size.
- Problems:
  - Complex algorithms to get aligned and contiguous physical frames.
  - Can only replace ~hundreds of contiguous smaller pages.
  - Internal fragmentation, higher disk traffic (single dirty bit), etc., etc.

Defining Contiguity

Contiguity exists when consecutive virtual pages are allocated consecutive physical page frames.

Contiguity in Huge Pages

- Let base page size be 4KiB. A 2MiB page requires an instance of:

$$\frac{2\text{MiB}}{4\text{KiB}} = 512\text{-page contiguity}$$

The Spectrum of Contiguity

- Very low contiguity: virtual pages and physical frames aren't contiguous at all.
- ...?
- ...?
- ...?
- Very high contiguity (~hundreds of base pages): just construct a superpage for maximal TLB reach.

The Spectrum of Contiguity

- Very low contiguity: virtual pages and physical frames aren't contiguous at all. (Standard TLB usage.)
- …?
- Intermediate contiguity (~tens of base pages): Generated by buddy allocators/memory compaction. CoLT exploits this.
- …?
- Very high contiguity (~hundreds of base pages): OS constructs a superpage for maximal TLB reach.

Subsection 4

Understanding Sources of Contiguity

Buddy Allocation

- Tracks free physical frames and assigns them to virtual pages on-demand.
- Uses a method based on binary trees to maximize contiguity.

Buddy Allocation: At a High Level

- Maintains K free-lists.
- `std::vector<std::list<pfn>> B(K)`

Buddy Allocation: At a High Level

- Maintains K free-lists (`B`).
- A free-list at index $x \in [0, K)$ tracks chunks of free physical memory of size 2^x pages.

Buddy Allocation: At a High Level

- Let a program request N pages worth of memory.
- Then, we search the free list at index $i = \lceil \log_2 N \rceil$.
- If `B[i]` has a chunk, give it to the program.
- Otherwise, walk up `B`, looking for free chunks.
  - Once found, keep splitting it in half until you get the size you want, and insert the remaining chunks into the appropriate free-list.

Buddy Allocation: At a High Level

- When physical pages are freed, the buddy allocator re-merges halves and moves them up to a higher size free-list.
- Obviously doing this in C++ with linked lists would be inefficient[2], kernel uses a bitmap to represent the binary tree for buddy tracking and uses efficient XORs to compute buddy addresses.

[2] Setting aside the fact that it's a terribly bloated language that has no direction ;).

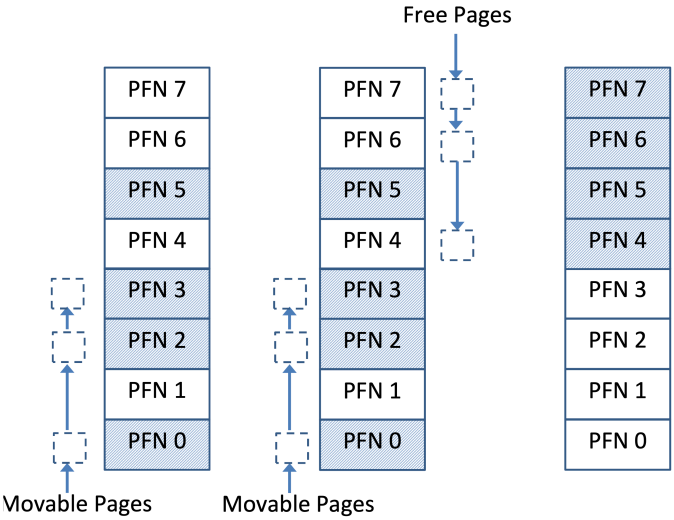
Anyway, the Point Is This...

- By construction, the buddy allocator *allocates contiguous physical pages* when a program requests memory that's larger than a single page.
- Unfortunately, the contiguity is on the order of tens of pages, so superpages can't be used.

Memory Compaction

- Systems running lots of processes (large working set) have lots of external fragmentation.
- For the buddy allocator to even have contiguous frames to track, we may have to move pages around to get a chunk of contiguous memory.
- On Linux, the `kcompactd` daemon does this job.

Memory Compaction: Illustration



Again...

- The memory compaction algorithm in tandem with the buddy allocator *naturally produces intermediate contiguity*.
- Greater system load actually increases contiguity since it triggers the compaction daemon more often.
- Linux's THS (transparent huge pages) often defaults to the buddy allocator breaking apart a (say, 2MiB) superpage into chunks – creating even more intermediate contiguity.

Section 2

Exploiting Intermediate Contiguity: CoLT (Coalesced Large-Reach TLBs)

Subsection 1

Design and Implementation

The Paper

Binh Pham, Viswanathan Vaidyanathan, Aamer Jaleel, and Abhishek Bhattacharjee. 2012. CoLT: Coalesced Large-Reach TLBs. In Proceedings of the 2012 45th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-45). IEEE Computer Society, USA, 258–269.
https://doi.org/10.1109/MICRO.2012.32

Principles of Coalescing

- Detect instances of consecutive virtual-to-physical translations.

Principles of Coalescing

- Detect instances of consecutive virtual-to-physical translations.
- Coalesce only on *misses* (not on hits to minimize lookup latency).

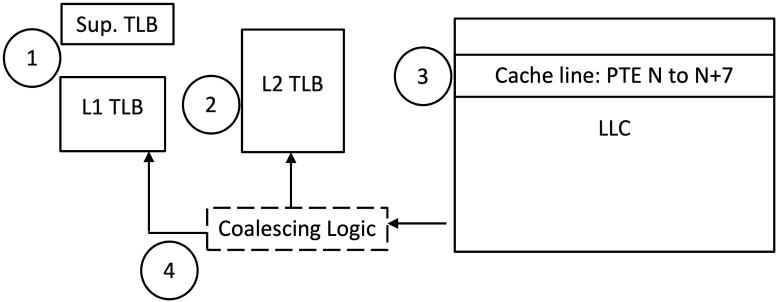
Principles of Coalescing

- Detect instances of consecutive virtual-to-physical translations.
- Coalesce only on *misses* (not on hits to minimize lookup latency).
- Unintrusive: this is as opposed to prefetching or speculation, which can degrade performance due to a misprediction.

Principles of Coalescing

- Detect instances of consecutive virtual-to-physical translations.
- Coalesce only on *misses* (not on hits to minimize lookup latency).
- Unintrusive: this is as opposed to prefetching or speculation, which can degrade performance due to a misprediction.
- Since temporal contiguity != system contiguity, CoLT ensures that at worst, a coalesced entry is unused via coalescing strictly on-demand.

High Level Operation (CoLT-SA)



High Level Operation (CoLT-SA)

- We want to load from a virtual address.

High Level Operation (CoLT-SA)

- We want to load from a virtual address.
- (1) In parallel, look up L1 and L2 TLBs. (2) Both missed.

High Level Operation (CoLT-SA)

- We want to load from a virtual address.
- (1) In parallel, look up L1 and L2 TLBs. (2) Both missed.
- (3) Page walk, get the PTE into LLC.

High Level Operation (CoLT-SA)

- We want to load from a virtual address.
- (1) In parallel, look up L1 and L2 TLBs. (2) Both missed.
- (3) Page walk, get the PTE into LLC.
- (4) Now, return the translation to the processor.

High Level Operation (CoLT-SA)

- We want to load from a virtual address.
- (1) In parallel, look up L1 and L2 TLBs. (2) Both missed.
- (3) Page walk, get the PTE into LLC.
- (4) Now, return the translation to the processor.
- (4) In parallel, the *Coalescing Logic* looks at translations around the new entry for contiguity, and tries to merge as many as possible,

High Level Operation (CoLT-SA)

- We want to load from a virtual address.
- (1) In parallel, look up L1 and L2 TLBs. (2) Both missed.
- (3) Page walk, get the PTE into LLC.
- (4) Now, return the translation to the processor.
- (4) In parallel, the *Coalescing Logic* looks at translations around the new entry for contiguity, and tries to merge as many as possible, *assuming they belong to the same set*.

Set Associativity is Tricky

- We assumed the the L1/L2 TLB was set-associative.
- Set indexing logic causes consecutive translations to be in consecutive sets – bad for coalescing.
- Change the indexing scheme.

Note

- The *Coalescing Logic* is off the critical path for execution since we can do it in parallel with returning the translation to the processor.
- It therefore doesn't affect how long a TLB miss takes.

Contiguous Set Indexing

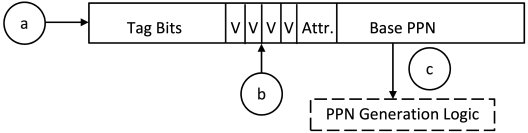
- Usually, the lowest bits of the VPN are used to index the set.
- This maps consecutive virtual pages to consecutive sets, precluding coalescing.

Contiguous Set Indexing

- Instead, to coalesce N entries, simply left-shift the index bits by $\lceil log_2 N \rceil$.
- This ensures that consecutive VPNs will map to the same set.
- Tradeoff: more and more conflict misses if we use higher and higher order bits.

Lookup

Coalesced TLB entry:



- (a) Match tags (upper bit of VPN).
- (b) Check valid bit for our specific VPN – index using lower (excluded) bits of the VPN.
- (c) Add on our offset to the base PPN to get the true non-coalesced PPN.

Constraints

- Set indexing issues.
- To prevent additional page table walks to check for contiguous entries to coalesce, CoLT-SA only checks the fetched cacheline for coalescing opportunities.
- For an 8B PTE and a 64B, this means we can coalesce at most 8 entries.

Replacement Policy and Invalidation

- The paper assumes LRU.
- However, acknowledges that a more complex scheme that prioritizes entries with more coalescing using attribute bits would be useful.
- It also mentions uncoalescing entries on invalidation to invalidate only the victim.

Overhead: Pretty Low

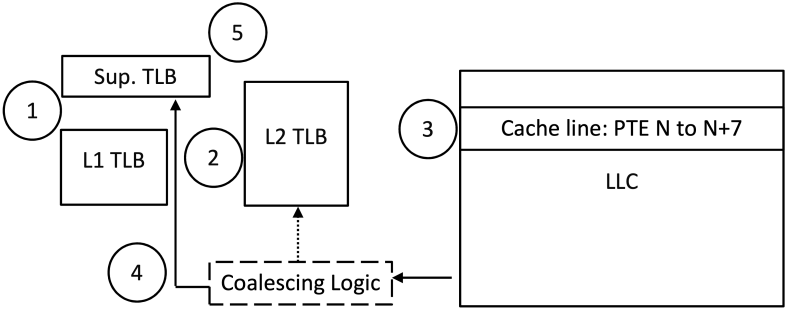
- Tag matching, simple logic.
- Valid matching, simple logic.
- PPN generation: requires an adder, but it's small since it only has to add offsets of up to N, where N is the number of coalesced entries.

CoLT-FA

- Uses the fully associative superpage TLB instead of the L1 TLB.[3]
- This gets rid of the set indexing issue.
- Doesn't compete with superpages for residency:
  - They're not used often.
  - When used, the accesses are to superpages are frequent, so they stay at the head of the LRU list.

[3] Note L2 is inclusive of both.

CoLT-FA: Operation

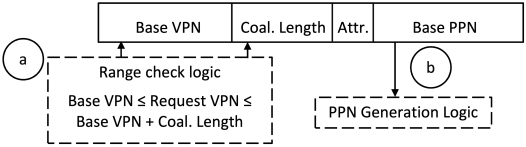


CoLT-FA: Operation

- (1, 2) Miss in all TLBs, (3) Page walk.
- (4) Coalesce all possible entries in cacheline.
- (5) “Scan” FA cache for more coalescing opportunities with a resident.

CoLT-FA: Lookup

FA cache TLB entry:

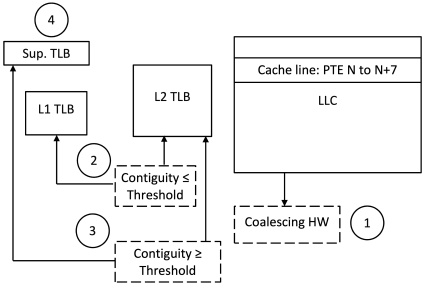


- (a) Check TLB hit: is the VPN in the range coalesced?
- (b) PPN generation: subtract off the base VPN from the requested VPN to get the offset, then add it to the base PPN.

Hardware Overhead

- More than CoLT-SA.
- Needs adders and comparators.
- More overhead from the scan in step 5 to coalesce with resident entries.

CoLT-All



- Combines CoLT-SA and CoLT-FA.
- Checks the amount of contiguity in the cacheline.
- If it is below a threshold, use SA, else use FA.

Subsection 2

Benchmarks

This Paper Had Tremendous Impact

- AMD uses coalesced TLBs in its flagship Zen architecture.
- Between 2017 and end-2019, AMD shipped over 260 million Zen cores with coalesced TLBs.
- By end-2022, AMD shipped an estimated one billion Zen cores with coalesced TLBs.

Back to the Paper...

- Test in simulation using various settings on the kernel:
  - On/off `defrag` flag.
  - On/off TPS.
  - Off/25%/50% `memhog`, a util that causes fragmentation on purpose.
- 12 possible configs, the paper picks 7.

Configs

First one is Linux default.

THS	`defrag`	`memhog`
Yes	Yes	No
Yes	Yes	25%
Yes	Yes	50%
No	Yes	No
No	Yes	25%
No	Yes	50%
No	No	No

Simulation Methodology: Part 1

- Trace-based analysis of TLB miss rates.
- Simics simulation of x86 processor, patched kernel to provide page walk details for all memory derefeneces: VPN, PPN, attribute bits.

Simulation Methodology: Part 1

- Run traces through custom memory simulator: 128-entry L1 and L2 4-way set-associative TLBs, 8-entry FA superpage TLB, 22-entry MMU cache.
- Use this to compute miss-rate elimination via CoLT.

Simulation Methodology: Part 2

- Finally, use Pin-based CMP$im to model a processor.
- (Processor is 4-way OOO w/128-entry ROB.)
- This is too slow to run full Linux distros along with memory allocation behavior necessary for CoLT (large, long-running apps).

Simulation Methodology: Interpolation

- However, using CMP$im to merely track allocated virtual pages along with our miss-rate elimination from Part 1, we can interpolate CoLT's performance gains.

Interpolation is Valid Because

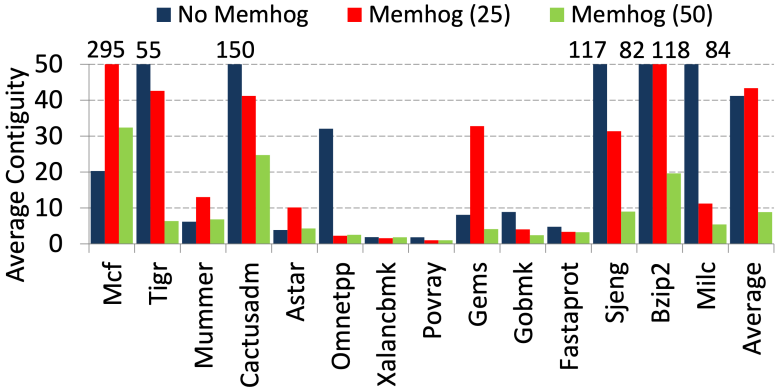
- TLB misses are on the execution critical path since the miss penalties are serialized.
- Interpolation doesn't account for instruction replay due to TLB misses, so the estimated benefit from CoLT is conservative.

Results

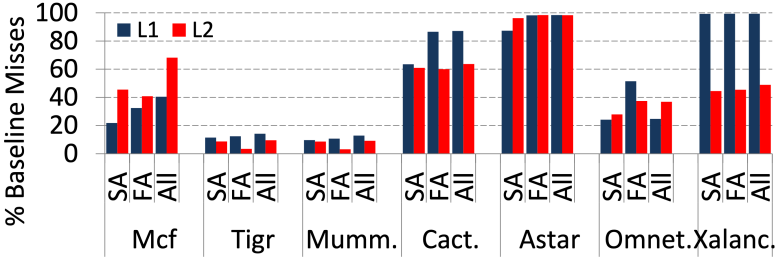
- The paper goes on to show using Part 2 that for various kernel configs, *intermediate contiguity* is *always* maintained.
- It uses Part 1 to analyze miss-rate elimination.
- Then it interpolates.
- Let's see some pictures!

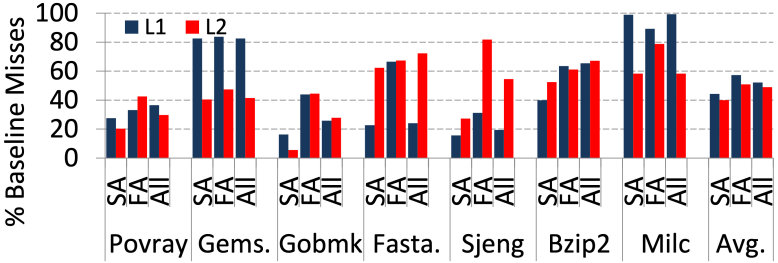
Average Contiguity

(Default: THS on, `defrag` on)



% Baseline Misses Eliminated





That's all.

Thanks for coming!